processors and their corresponding caches so that each processor has the latest value for `counter` when it does the increment. If `do_something()` is short enough, or if there are too many processors running this code, the processors might actually find themselves *waiting* for each other; one processor is ready to update the value, but another processor is currently doing that, so it has to wait until the second processor has completed its update and the change has propagated. This situation is called *high contention*. If the processors rarely have to wait for each other, you have *low contention*.

In a loop like this one, the data for `counter` will be passed back and forth between the caches many times. This is called *cache ping-pong*, and it can seriously impact the performance of the application. If a processor stalls because it has to wait for a cache transfer, it can't do *any* work in the meantime, even if there are other threads waiting that could do useful work, so this is bad news for the whole application.

You might think that this won't happen to you; after all, you don't have any loops like that. Are you sure? What about mutex locks? If you acquire a mutex in a loop, your code is similar to the previous code from the point of view of data accesses. In order to lock the mutex, another thread must transfer the data that makes up the mutex to its processor and modify it. When it's done, it modifies the mutex again to unlock it, and the mutex data has to be transferred to the next thread to acquire the mutex. This transfer time is *in addition to* any time that the second thread has to wait for the first to release the mutex:

```
std::mutex m;
my_data data;
void processing_loop_with_mutex()
{
    while(true)
    {
        std::lock_guard<std::mutex> lk(m);
        if(done_processing(data)) break;
    }
}
```